

Feedback from Saqlain (After Submission)

fathom.video
https://fathom.video/share/q7AtsfvcGZAUnUb74GgMZCJ_zNgPboaA

1. GHL Push from Inventory (People table only)

1a. Clients Table (Supabase)

Create a new clients table in Supabase:

```
CREATE TABLE clients (  
  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
  slug        TEXT UNIQUE NOT NULL,  
  
  name        TEXT NOT NULL,  
  
  ghl_api_key  TEXT,  
  
  ghl_location_id  TEXT,  
  
  emailbison_api_key  TEXT,  
  
  emailbison_workspace_id TEXT,  
  
  is_active    BOOLEAN DEFAULT true,  
  
  updated_at   TIMESTAMPTZ DEFAULT NOW()  
  
);
```

Populate with all 12 clients. Credentials are in the project .env file — map GHL_<CLIENT>_API_KEY, GHL_<CLIENT>_SUBACCOUNT_ID, EMAILBISON_<CLIENT>_KEY to the right rows.

1b. Platform Pushes Table (Supabase)

```
CREATE TABLE platform_pushes (  
  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
  person_id   UUID REFERENCES people(id) ON DELETE CASCADE,
```

```

client_id      UUID REFERENCES clients(id),

platform      TEXT NOT NULL,

platform_contact_id TEXT,

campaign_tag   TEXT,

pushed_at     TIMESTAMPTZ DEFAULT NOW(),

UNIQUE(person_id, client_id, platform)

);

```

1c. Clients Management Page in Interface

Add a Clients page to the interface. Simple table of all clients with inline-editable credential fields. Saving updates the clients table in Supabase. This is how we switch sub-account IDs or API keys for any client without touching code.

1d. Push to GHL Flow

Trigger: User is on the People page with filters applied. They click "Push to GHL."

Step 1 — Select client. Dropdown populated from clients table (is_active = true).

Step 2 — Fetch GHL custom fields for that sub-account.

GET <https://services.leadconnectorhq.com/locations/{locationId}/customFields>

Authorization: Bearer {ghl_api_key}

Version: 2021-07-28

Cache per sub-account per session. Do not call per contact.

Step 3 — Custom field mapping (if virtual enrichment columns are active). Show mapping UI: "Your enrichment field → GHL custom field." User maps or skips each one.

Step 4 — Push contacts. For each person in the filtered list:

Build the structured tag:

{Client Name} - {Niche} | {employee_range} | {country} | {data_source}

- Employee ranges: 1–10, 11–50, 51–200, 201–500, 500+
- Country from [companies.country](#)

- Data source from companies.source — use as-is (comma-separated if multi-source)

POST to GHL:

POST <https://services.leadconnectorhq.com/contacts/>

Authorization: Bearer {ghl_api_key}

Version: 2021-07-28

```
{  
  
  "locationId": "{ghl_location_id}",  
  
  "firstName": "...",  
  
  "lastName": "...",  
  
  "email": "...",  
  
  "phone": "...",  
  
  "companyName": "...",  
  
  "city": "...",  
  
  "country": "...",  
  
  "tags": ["{structured_tag}"],  
  
  "customField": [{"id": "{field_id}", "value": "..."}]  
  
}
```

Duplicate handling: If GHL returns 400 with meta.contactId, treat as soft success. Append the tag to the existing contact:

POST /contacts/{contactId}/tags

```
{"tags": ["{structured_tag}"]}
```

Rate limit: 200ms between calls.

Step 5 — Write back to Supabase.

```
INSERT INTO platform_pushes (person_id, client_id, platform, platform_contact_id, campaign_tag, pushed_at)
VALUES (...)

ON CONFLICT (person_id, client_id, platform) DO UPDATE

SET platform_contact_id = EXCLUDED.platform_contact_id,

    campaign_tag = EXCLUDED.campaign_tag,

    pushed_at = EXCLUDED.pushed_at;
```

Also update people.pushed_to_ghl = true and pushed_to_ghl_at = NOW().

Step 6 — Push summary.

Sub-account: Kynship

Total selected: 1,200

Created (new): 980

Tag-appended: 214

Failed: 6

Default filter: Only push phone_type IN ('mobile', 'toll_free'). Skip landlines.

2. EmailBison Push from Inventory (Companies table + People table)

Follow the exact same pattern as Clay's native EmailBison integration. Clay has two built-in enrichments: one to add a lead to an EmailBison workspace, one to add them to a campaign. Saqlain will walk you through how those work on the call. Replicate the same API calls in the inventory interface.

Credentials come from the clients table (emailbison_api_key, emailbison_workspace_id).

Variable names: EmailBison custom variables do not require fetching IDs. Just pass the variable name exactly as it appears in the EmailBison workspace.

Duplicate handling: If EmailBison returns 422 "already taken" on an email, treat as soft success and continue.

Write back to platform_pushes same as GHL, with platform = 'emailbison'.

Placement: Add the Push to EmailBison button to both the Companies table and the People table.

3. Custom Data Filtration (Virtual Columns)

Enrichment fields live in custom_data JSONB and are different for every campaign. The solution is virtual columns — they exist only in the interface, not in the database schema.

How it works

Adding a virtual column:

1. User clicks "Add column from enrichment" on the Companies or People table.
2. Interface fetches all distinct keys in custom_data across the current filtered result set:

```
SELECT DISTINCT jsonb_object_keys(custom_data)
```

```
FROM people
```

```
WHERE [current filters]
```

```
ORDER BY 1;
```

1. User picks a key (e.g. lead_score) and selects its type: Text / Number / Boolean.
2. Column appears in the table reading custom_data->>'lead_score' per row.
3. Filter controls appear based on type:
 - **Number:** is / is not / greater than / less than / between
 - **Text:** is / is not / do not contain / contains / is empty / is not empty
 - **Boolean:** is true / is false

Backend filter queries:

```
-- Number between
```

```
WHERE (custom_data->>'lead_score')::numeric BETWEEN 90 AND 100
```

```
-- Text equals
```

```
WHERE custom_data->>'ecom_qualification' = 'qualified'
```

```
-- Text contains
```

```
WHERE custom_data->>'ecom_qualification' ILIKE '%qualified%'
```

```
-- Is not empty
```

```
WHERE custom_data->>'lead_score' IS NOT NULL
```

AND custom_data->>'lead_score' != "

-- Key exists

WHERE custom_data ? 'lead_score'

Column state is frontend-only. Nothing is written to the Supabase schema. custom_data JSONB is never modified by this feature.

After push or export: Interface prompts: "You used these temporary columns: lead_score, ecom_qualification. Remove them from view?" User selects which to remove. They disappear from the table. Underlying data in custom_data is untouched.

Virtual columns + GHL mapping: If virtual columns are active when the user initiates a GHL push, the custom field mapping step picks them up automatically and offers to map them to GHL custom fields.

4. Speed Optimization

Audit and optimize the full interface for speed. Focus areas: query efficiency, indexes, pagination, unnecessary re-fetches on filter change, large payload sizes. Estimated time: 1 day.

5. UI Fixes

5a. Filter dropdown scrolling bug. On Companies and People pages, scrolling while a filter dropdown is open scrolls the main results list instead of the options within the dropdown. Fix: give the dropdown options list its own scroll container with overflow-y: auto and a fixed max-height. Lock main page scroll while a dropdown is open.

5b. Remove "Re-verify Phone" from Companies table only. Remove the Re-verify Phone button from the Companies table. Keep Re-verify Email on the Companies table. Keep both buttons on the People table.

5c. Add Job Title column to People table. In the People list view, add a Job Title column immediately after Full Name. Field is job_title on the people table — already in the database, just needs mapping into the list view.